基于深度学习模型的图书推荐系统

2021111624 王思涵

目录

1 任务意义和背景 **2**
1.1 数据来源与数据介绍 2
1.2 研究意义与研究背景 2

2 所用深度学习方法介绍 **2**
2.1 深度因子分解机 (DeepFM) 2
2.2 注意力因子分解机 (AFM) 3
2.3 深度交叉网络 (DCN) 5
2.4 自动特征交叉学习模型 (AutoInt) 6
2.5 深度兴趣网络 (DIN) 7
2.6 双重样本感知的因子分解机 (DIFM) 9

3 数据预处理 **10**
3.1 划分训练集与测试集 10
3.2 负样本构建 11
3.3 数据加载器使用 11

4 模型训练 **11**
4.1 Deep Factorization Machine 11
4.2 Attention Factorization Machine 14
4.3 Deep Cross Network 16
4.4 Automatic Feature Interaction Learning 17
4.5 Deep Interest Network 19
4.6 Dual Input-aware Factorization Machine 21

5 模型评估 **23**
5.1 准确率、精确率、召回率、F1 分数 23
5.2 混淆矩阵 23

6 反思总结 **23**
6.1 负样本构建 23
6.2 超参数选择 24
6.3 评价指标 24

1 任务意义和背景

1.1 数据来源与数据介绍

本实验的数据集选自 DataFountain 竞赛平台的隐式用户图书交互数据集，相关链接为https://www.datafountain.cn/competitions/542/datasets。

该数据集由 5869631 条记录行，2 个数据特征列组成。数据说明如下：

（1）user_id：用户编号

（2）item_id：图书编号

其中共 53424 位用户，10000 本图书。每条数据行记录代表该用户对该图书做出评分行为，是一条已经存在的用户图书交互记录。通过对数据集的直观观察，我们能够得出该数据集数据极度稀疏并且能够使用的特征有限，用户特征和物品特征各只有一个。

1.2 研究意义与研究背景

随着新型互联网的发展，人类逐渐进入了信息爆炸时代。新型电商网络面临的问题也逐渐转为如何让用户从海量的商品中挑选到自己想要的目标。推荐系统正是在互联网快速发展之后的产物。随着我国经济的快速发展和物质生活水平不断提高，人们的精神文化生活日益丰富，阅读已成为人们生活中不可或缺的一部分。手机用户不只是网站内容消费者，同时也是数据生产者。人们不管在何时何地，都可借助移动终端在网络平台上进行评分从而产生互动数据集。

为帮助电商系统识别用户需求，为用户提供其更加感兴趣的信息，从而为用户提供更好的服务，需要依据真实的图书阅读数据集，利用机器学习的相关技术，建立一个图书推荐系统。用于为用户推荐其可能进行阅读的数据，从而在产生商业价值的同时，提升用户的阅读体验，帮助创建全民读书的良好社会风气。

2 所用深度学习方法介绍

2.1 深度因子分解机 (DeepFM)

一、DeepFM 简述

DeepFM(Deep Factorization Machine) 模型是 2017 年由华为若亚方舟团队提出来的一种新型 NN 模型，由于 FM 模型可以解决低阶特征组合却无法解决高阶特征挖掘的问题，所以结合谷歌公司提出的 Wide&Deep 模型对其改进，将其 Wide 中 logistic 回归部分更新为 FM 与 DNN 进行交叉，使得框架可以直接从原始特征中同时学习低阶特征和高阶特征，有效地提高了训练效率，从而引入 DeepFM 模型。

DeepFM 是集成 FM 和 Deep 的一个神经网络框架，DeepFM 与 Wide&Deep 不同的地方在于 DeepFM 可以进行端到端的方式对特征交叉训练，无需在原始特征上做任何特征工程训练。该算法提出后在 banchmark 数据和商业数据上都进行了评测，对现有模型训练效率真实有效地进行了提升，并被工业界大量用于广告点击预估和推荐系统使用，并且都有非常不错的效果，所以被越来越广泛的使用。

二、DeepFM 具体流程

（1）使用 FM 模型进行 Embedding 训练，得到 Dense Embeddings 的输出。

（2）将 Dense Embeddings 结果再次输入 DeepFM 模型中的 FM 模块和 DNN 模块，通过一定方式组合，FM 模块模拟出 FM 的效果，DNN 模块训练出比 FM 模块更高阶的特征交叉。

（3）将 DNN 和 FM 的训练结果组合后激活输出。

三、FM 部分

DeepFM 模型在构建 FM 部分时在线性部分对特征分配权重再进行加权求和，之后在交叉部分对用户特征和图书特征进行两两交叉相乘并赋予权重，然后进行加权求和，之后将两部分的结果叠加作为 FM 层的输出。FM 部分的公式见下式。

$$y(x) = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} < v_i, v_j > x_i x_j \tag{1}$$

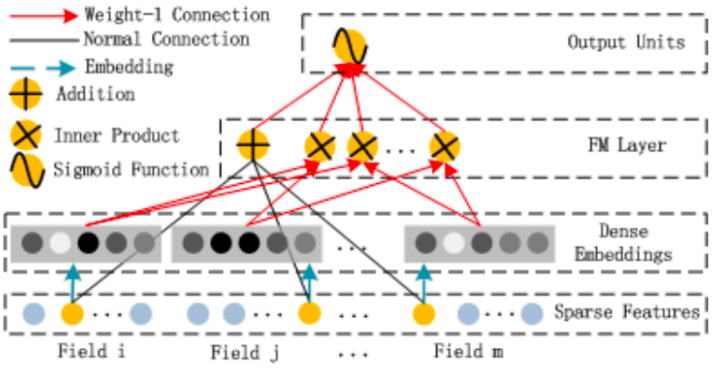

图 1: FM 部分结构

四、Deep 部分

在 Deep 部分将用户特征和视频特征做 Embedding 特征交叉再拼接原有数据，作为 Deep 部分的输入。之后通过多层线性和非线性映射得到隐藏层的输出，之后通过多层线性和非线性映射得到隐藏层的输出如下式。

$$\begin{aligned} a^{(0)} &= [e_1, e_2, ..., e_m] \\ a^{(i+1)} &= \sigma(W^{(i)} a^{(i)} + b^{(i)}) \end{aligned} \tag{2}$$

a^0 为 embedding 层输出，e_i 表示第 i 个 field 的 embedding，m 是 field 的数量。σ 为激活函数，a、W、b 为该层的输入、权重、偏置。Deep 部分将最终结果与 FM 结果累加得到最终视频预测结果。

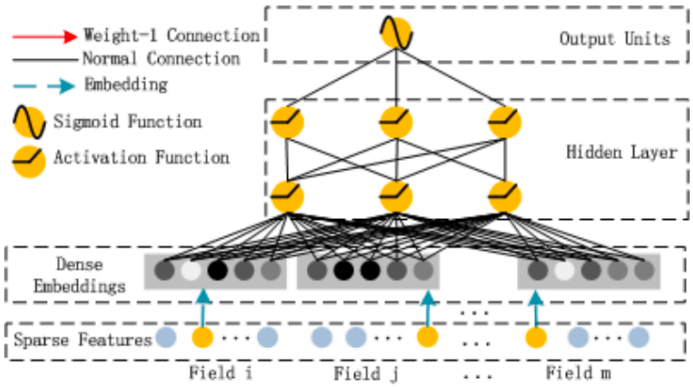

图 2: DNN 部分结构

DeepFM 模型输出结果如下式。

$$\hat{y} = sigmoid(y_{FM} + y_{DNN}) \tag{3}$$

其中 y_{FM} 是 FM 的输出结果，y_{DNN} 是 DNN 的输出结果，使得低阶特征和高阶特征累加融合，再经过 sigmoid 函数非线性转换输出点击预估结果。

2.2 注意力因子分解机 (AFM)

一、AFM 简述

AFM(Attention Factorization Machine) 模型在 2017 年被提出，是由浙江大学和新加坡国立大学共同开发的。FM 算法使用二阶特征能提高线性模型性能，它假设交叉特征都具有相同的权重，但在应用中发现每个交叉特征的影响程度并不一样，并且存在无关的特征容易给模型学习产生不利影响。因此，AFM 模型的关键是在于用来得到不同交互特征的贡献度，它在常规的回归运算中，比 FM 模型的效率更高，且更加简单。

监督学习属于机器学习的基本任务之一，它的目标是判定一个函数是否可以通过给定特征作为输入，来生成一个预测的结果。当对分类预测变量进行监督学习时，考虑它们之间的交互作用是很重要的。若想要得到特征之间的交互，一种常用且较为有效的方法是使用显式特征，比如在多项式回归中，通过学习获得各数据集属性中交叉特性的权值。然而，为了避免多项式回归的泛化问题，提出了因子分解机，它将各个特征的权重系数转化为嵌入向量的内积，并对其进行了分析。AFM 则是在 FM 基础上进行改进，它同样也是在注意力机制上改进的神经网络模型，使得特征间的相互作用在预测中起到了不同的作用，从而使得预测更为精确。

二、AFM 结构

注意力因子分解机从交互式特征对于目标预测的重要意义入手，在经典的因子分解机模型的基础上，加入多个注意力机制，使各交互式特征之间的权值由多层感知器来获得，从而实现对目标的预测。AFM 数学表达式为下式。

$$y_{AFM} = w_0 + \sum_{i=1}^{n} w_i x_i + P^T \sum_{i=1}^{n} \sum_{j=i+1}^{n} a_{ij} < v_i, v_j > x_i x_j \tag{4}$$

AFM 模型由输入层、嵌入层、特征交互网络、注意力网络和输出层组成。模型的输入层是稀疏特征向量，通过嵌入层将这些稀疏向量映射到稠密向量空间中，得到输入向量对应的

稠密向量，然后将稠密特征向量进行两两交叉相乘，接着将交叉特征输入到注意力网络中，得到每一对交叉特征向量的权重，最后通过加权求和得到最终的输出。

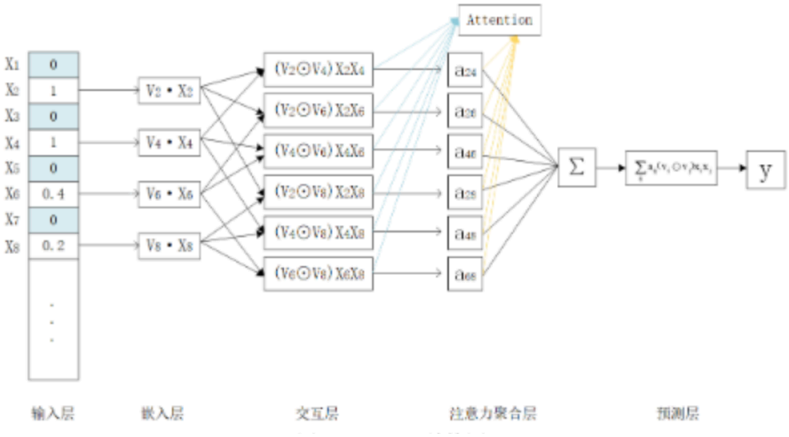

图 3: AFM 模型结构

三、注意力网络

在 (4) 公式中，a_{ij} 表示的是相互作用向量的注意力分数，也就是相互作用向量对于所预测对象的重要性影响程度大小。注意力网络的目标是利用多层感知器对权值进行学习，其表达式为下式。

$$
\begin{aligned}
a_{ij}^{'} &= h^T ReLU(W(< v_i, v_j >)x_i x_j + b) \\
a_{ij} &= \frac{exp(a_{ij}^{'})}{\sum_{(i,j)\in R_x} exp(a_{ij}^{'})}
\end{aligned}
\tag{5}
$$

2.3 深度交叉网络 (DCN)

一、DCN 简述

DCN(Deep Cross Network) 模型最初由华为诺亚方舟实验室提出，用于解决 CTR 预估问题。在 DCN 模型中，每个特征都会被送入交叉网络中，经过交叉处理后，输出的结果会经过多层神经网络的处理，并输出预测结果。由于 DCN 模型能够自动地进行特征交叉，同时具有较好的可解释性和泛化能力，因此被广泛应用于 CTR 预估、推荐系统等领域。它引入了 Cross Network 和 Deep Network 两个部分，使得模型能够学习到高阶特征的交叉信息和低阶特征的深层次抽象表示。DCN 是一种结合了线性模型和神经网络模型的深度交叉网络模型。它使用了 cross network，在每一层都应用特征交叉，从而高效的学习了 bounded degree 组合特征。Cross Network 通过将特征的交叉信息传递到不同层级的网络中，有效地学习到了高阶交叉特征；而 Deep Network 则通过多个全连接层进行特征的深层次表示学习。这个特点使得 DCN 在特征交叉的处理上比传统的 DNN 模型更加高效，而且参数更少，具有更好的性能表现。DCN 模型的网络结构相对简单，多项式复杂度由层次深度决定。相比于 DNN，DCN 的 logloss 更低，而且参数的数量将近少了一个数量级。

二、Cross Network

DCN 模型的核心是 Cross Network，也称为 Cross Layer，它是一种特殊的神经网络层，用于自动地学习输入特征的高阶交叉项。Cross Network 通过对输入特征进行两两交叉，产生

新的高阶交叉特征，并使用全连接层进行线性组合，最终输出交叉特征的加权和。在 Cross Network 中，不同的特征交叉项拥有独立的权重，并且不同特征交叉项的权重可以共享。

三、Deep Network

除了 Cross Network，DCN 模型还包括 Wide Network 和 Deep Network 两部分。Wide Network 用于学习输入特征的低阶交叉项，并且可以捕捉特征之间的共现关系，例如某些特征经常同时出现。Deep Network 则用于学习特征的抽象表达，通过多个隐层的神经网络对输入特征进行非线性变换，提取更高层次的特征表示。最后，Wide Network 和 Deep Network 的输出会经过一个融合层，得到最终的 CTR 预估结果。

总之，DCN 模型通过 Cross Network 对输入特征进行高阶交叉，通过 Wide Network 学习低阶交叉特征，通过 Deep Network 学习高阶抽象特征，最终通过融合层得到 CTR 预估结果，具有较好的性能和可解释性。

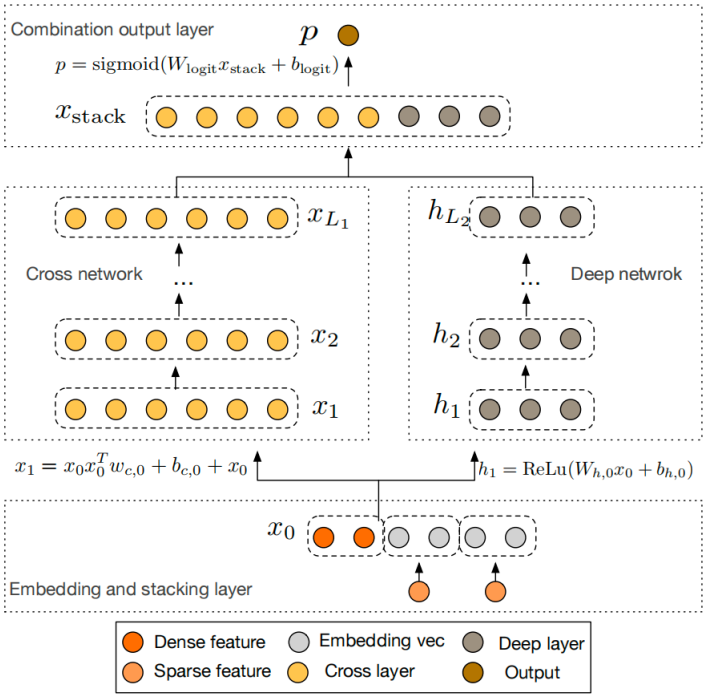

图 4: DCN 模型结构

2.4 自动特征交叉学习模型 (AutoInt)

一、AutoInt 简述

随着深度学习在点击率预估领域的广泛应用，如何更好地对高阶交叉特征提取显得尤为重要。为了解决部分模型不具备良好的解释性并且交互效果不明显问题，北京大学团队基于多头自注意力机制提出 AutoInt(Automatic Feature Interaction Learning via Self-Attentive Neural Networks) 模型。AutoInt 自动学习特征之间的高阶交互关系，解决了人工处理的复杂性，更好地建模特征之间的关系，提高模型的预估准确性，同时处理稀疏和密集特征，从而能够应对更多的实际应用场景，可以扩展到多任务学习中，从而能够更好地适应各种应用场景，也可以通过增加注意力头的方式提升模型的性能。

Auto 模型包含嵌入层、交互层、输出层三个部分，其中嵌入层和输出和普通模型没什么区别，主要介绍一下交互层。

二、交互层 (Interacting Layer)

Interacting Layer 是 AutoInt 模型实现特征交叉的部分，其实现特征交叉的过程可以分为以下几步：

（1）计算特征之间的相关性

$$\alpha_{m,k}^{(h)} = \frac{exp\{\psi^{(h)}(e_m, e_k)\}}{\sum_{l=1}^{M} exp\{\psi^{(h)}(e_m, e_l)\}}$$
$$\psi^{(h)}(e_m, e_k) =< W_{Query}^{(h)} e_m, W_{Key}^{(h)} e_k > \tag{6}$$

其中 e_m、e_k 表示特征 m、k 对应的 embedding，$W_{Query}^{(h)}$、$W_{Key}^{(h)}$ 都是 $d' \times d$ 大小的矩阵，上标 h 表示第 h 个 head。

（2）根据相关系数，计算特征交叉

$$\tilde{e}_m^{(h)} = \sum_{k=1}^{M} \alpha_{m,k}^{(h)} (W_{Value}^{(h)} e_k),$$
$$where\ W_{Value}^{(h)} \in R^{d' \times d} \tag{7}$$

（3）拼接多个头的特征交叉结果

$$\tilde{e}_m = \tilde{e}_m^{(1)} \oplus \tilde{e}_m^{(2)} \oplus ... \oplus \tilde{e}_m^{(H)} \tag{8}$$

（4）为了保留原始特征，用一个残差结构拼接原始特征和交叉后的特征

$$e_m^{Res} = ReLU(\tilde{e}_m + W_{Res} e_m) \tag{9}$$

（5）最终在输出层，拼接所有交叉后的特征过 sigmoid 函数

$$\hat{y} = \sigma(w^T (e_1^{Res} \oplus e_2^{Res} \oplus ... \oplus e_M^{Res}) + b) \tag{10}$$

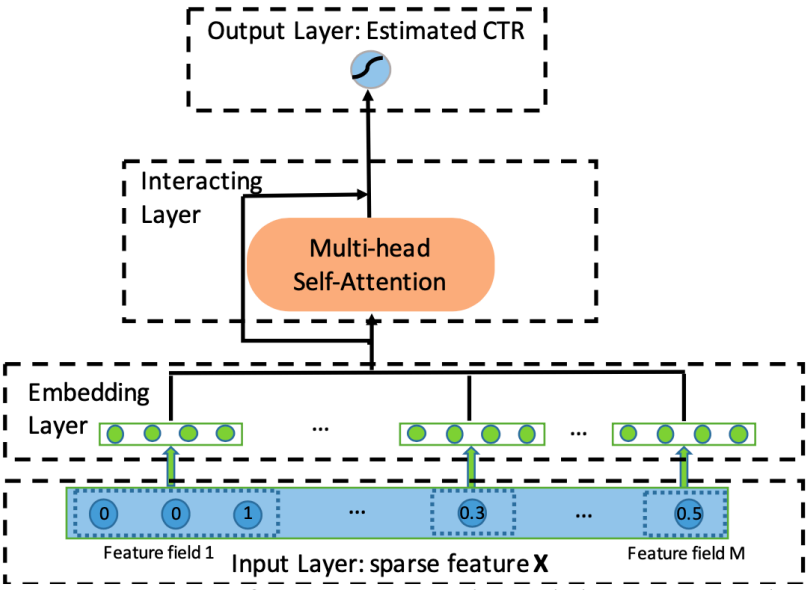

图 5: AutoInt 模型结构

2.5 深度兴趣网络 (DIN)

一、DIN 简述

DIN(Deep Interest Network) 网络，是阿里巴巴集团提出的一种用于提高推荐系统效率的深度学习框架。其发展历程如下：

（1）2017 年，阿里巴巴工程师首次提出 DIN 框架，该框架以减少推荐系统中的信息不一致性为目标。

（2）首次出现的 DIN 框架，仅基于用户的历史行为和与他们相关的商品信息，通过深度神经网络来建模和学习用户的兴趣。

（3）2018 年，DIN 框架得到了拓展，提出了一种新的方法：动态兴趣演化。该方法通过模拟用户的行为变化和商品的变化来预测用户兴趣的演化趋势并加以利用。

（4）2019 年，阿里巴巴发布了基于 DIN 框架的推荐系统算法——深度兴趣网络。DIEN 在 DIN 的基础上，进一步加强了用户的行为序列建模，包括动态兴趣演化，实现了更为精准的推荐。

（5）目前，DIN 框架已成为阿里巴巴推荐系统的核心算法之一，并在业界得到广泛的应用和推广。互联网企业中的许多推荐系统也采用了 DIN 框架及其拓展算法，例如淘宝、京东、蘑菇街等。

DIN 算法的核心思想是通过引入兴趣抽取网络来学习用户的兴趣表示。在兴趣抽取网络中，DIN 算法将用户的历史点击序列作为输入，并通过注意力机制来计算用户对每个兴趣的权重。这样，DIN 算法可以更好地捕捉用户的兴趣偏好，并在推荐时考虑到用户的多样性。

二、DIN 算法的关键组件

（1）兴趣抽取网络：该网络用于学习用户的兴趣表示。它通过将用户的历史点击序列输入到一个多层感知机中，并使用注意力机制来计算用户对每个兴趣的权重。

（2）候选广告池：该组件用于生成候选广告集合。在 DIN 算法中，候选广告池通常由用户的历史点击序列中的广告以及其他相关广告组成。

（3）广告排序网络：该网络用于对候选广告进行排序。它通过将用户的兴趣表示和候选广告的特征进行拼接，并通过多层感知机来学习广告的排序权重。

通过以上组件的组合，DIN 可以通过动态兴趣抽取和兴趣进化机制，能够更好地捕捉用户的兴趣演化和兴趣偏好。

三、DIN 模型中的注意力机制与兴趣进化机制

DIN 模型用到的公式主要是基于注意力机制和兴趣进化机制。

注意力机制公式如下式。

$$a_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{n_i} exp(e_{ik})} \tag{11}$$

其中，表示用户的兴趣 i 对物品 j 的注意力值，e_{ij} 表示用户的兴趣 i 对物品 j 之间的兴趣得分。

兴趣进化公式如下式所示。

$$v_i = \sum_{j=1}^{n_i} a_{ij} \cdot f(x_j) \tag{12}$$

其中，v_i 表示用户的兴趣 i 的进化向量，$f(x_j)$ 表示物品 j 的特征向量。

最终，DIN 模型的输出为下式。

$$y_{DIN} = W_1 \cdot [h_i, v_i] + b \tag{13}$$

其中，W_1 表示权重矩阵，h_i 表示用户 i 的隐层表示向量，v_i 表示用户 i 的进化兴趣向量。

相比传统的推荐算法，DIN 算法能够更好地解决用户兴趣多样性和长尾问题，提高推荐的准确性和用户满意度。DIN 框架和其拓展算法的应用情况显示出 DIN 框架在推荐系统领域具有较高的实用价值。

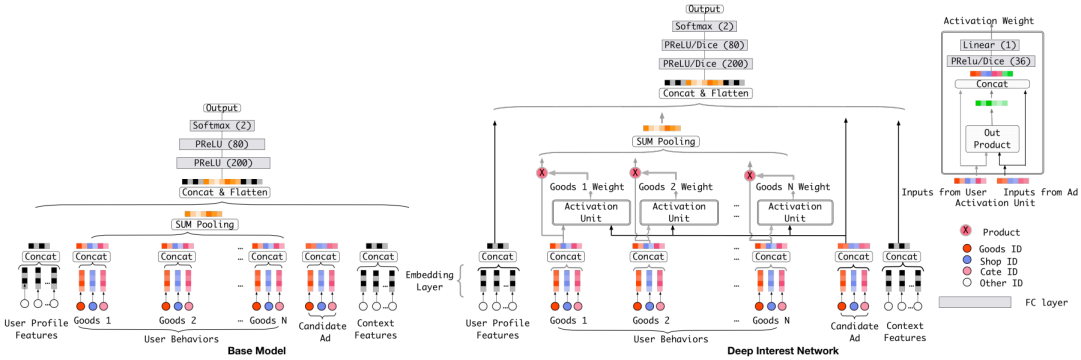

图 6: DIN 模型结构

2.6 双重样本感知的因子分解机 (DIFM)

一、DIFM 模型简述

DIFM(Dual Input-aware Factorization Machine) 模型是在 IFM 模型为基础上，加入了 transformer 中的主要思想，学习向量级别的交互后与元素级别的特征进行组合，学到最后的权重，对原始特征进行转化。IFM 模型提出的初衷是为了发现样本数据的独特性，模型认为对于不同的目标字段，同一个特征相对于不同的目标字段做预测的时候发挥的重要性不同。比如样本情况如下所示：当某个输入样本为 < 青少年，女性，学生，喜欢粉色 >，预测用户是否会点击 < 连衣裙 > 时，< 女性 > 这个特征在样本所有特征中发挥的作用就比较关键；当输入样本变化为 < 青少年，女性，学生，喜欢蓝色 >，预测是否会点击 < 笔记本 > 时，< 女性 > 这个特征就不如前面一个样本那么重要。IFM 通过 DNN 结构计算嵌入特征向量的元素级别的交互作用，但 DIFM 认为 DNN 是否是学习输入感知因子最有效的模型仍是一个悬而未决的问题，并且元素级别学习本身也不可能实现最佳预测。所以引入了一种新的矢量网络，这种网络在学习输入感知因子以调整原始特征表示的权重方面更有效，最终模型将多头自注意、残差网络和 DNN 等组件整合成统一的端到端模型。

二、向量部分的信息提取

论文的主要创新部分在 Dual-FEN Layer，这个模块包括向量部分和元素级别的信息提取，旨在学习不同输入实例中相同特征的唯一输入感知因子。

向量部分的输入矩阵 E_x 是经过嵌入向量重塑后的。

$$U_{vec} = reshape(E_x) \tag{14}$$

其中，矩阵大小为 $h \times k$，h 指字段的数量，k 指嵌入向量的大小。

将重塑后的输入向量通过多头自注意力机制的提取获得不同空间上的信息之后再连接起来，并且为保留嵌入向量的信息，论文使用残差网络将原特征嵌入向量添加到多头自注意力机制的输出中。具体实现公式如下式。

$$\begin{aligned} Q_i &= U_{vec} W^{Q_i} \\ K_i &= U_{vec} W^{K_i} \\ V_i &= U_{vec} W^{V_i} \end{aligned} \tag{15}$$

通过投影矩阵与输入向量 U_{vec} 相乘得到多头自注意力机制需要的三种输入类型即键、值、对，后通过下式的注意力计算公式得到每一个“头”的结果，最后将多个头相连接。

$$\begin{aligned} Attention(Q_i, K_i, V_i) &= softmax(\frac{Q_i K_i^k}{\sqrt{d_k}}) V_i \\ MulitHead(U_{vec}) &= Concat(head_1, head_2, ..., head_n) \\ head_i &= Attention(Q_i, K_i, V_i) \end{aligned} \tag{16}$$

$head_i$ 表示第 i 个头的输出，n 代表头的个数，$MulitHead(U_{vec}) \in R^{h \times (d_v \times n)}$ 是整个多头自注意力网络的输出，表示注意力因子的大小。

残差网络的表示如下式。

$$\begin{aligned} Readual(U_{vec}) &= Concat(R_1, R_2, ..., R_n) \\ R_i &= U_{vec} W^{R_i} \end{aligned} \tag{17}$$

$R_i \in R^{h \times d_r}$ 是与相关的第个残块。最后向量部分的输出如下式。

$$O(vector) = \sigma(Multihead(U_{vec}) + Resdual(U_{vec})) \tag{18}$$

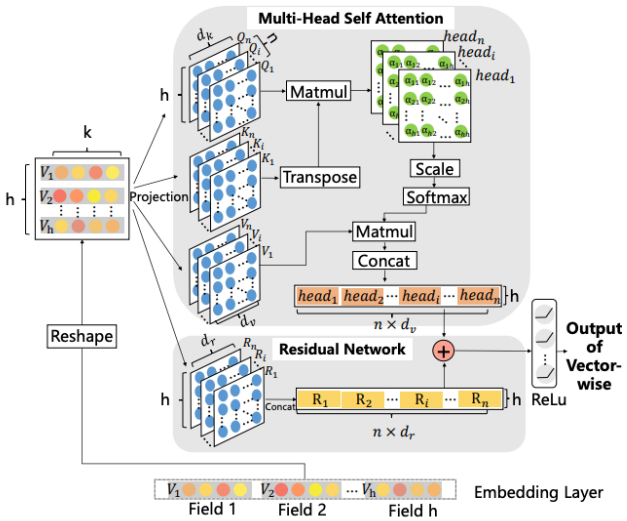

图 7: 向量部分信息提取结构

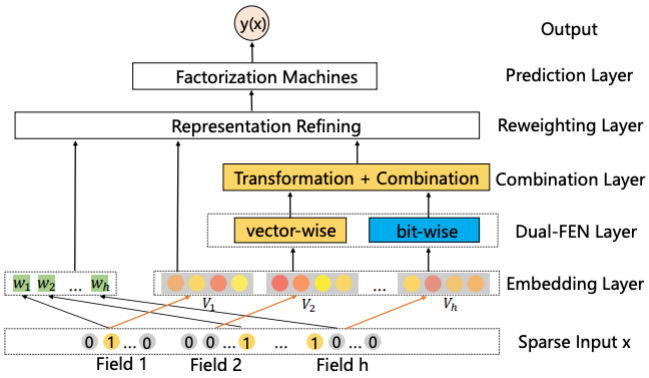

图 8: DIFM 模型结构

3 数据预处理

3.1 划分训练集与测试集

遵循序列性，按照用户交互记录的前 80% 作为训练集，后 20% 作为测试集。

3.2 负样本构建

由于已经数据集用户图书交互记录均为正样本，所以我们需要为根据训练集和测试集的用户未进行交互的图书部分构建负样本。

（1）训练集负样本构建：参考官方 baseline，为每个正样本随机生 3 个负样本。

（2）测试集负样本构建：由于我们需要使用准确率、召回率、F1 分数指标评价模型性能，所以使测试集中负样本比例远大于训练集中负样本比例。为每个正样本随机生成 40 个负样本。

3.3 数据加载器使用

由于数据集庞大，难以一次性转换为 pytorch 张量，故使用 dataloader 工具对数据进行分批读取，保证数据完整性。

4 模型训练

基础训练过程统一使用 Adam 优化器和二元交叉熵损失函数，训练轮次为 10。优化器和损失函数选择在部分模型超参数调整时会进行调整尝试。colab 和 kaggle 可用 GPU 资源有限，训练轮次已经在可用范围内尽力保证。

4.1 Deep Factorization Machine

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）前馈神经网络层层数：3

（3）前馈神经网络层各层维度 mlp_dims：[32, 16, 16]

（4）丢弃率 dropout：0.5

（5）学习率 lr：0.001

二、模型训练过程

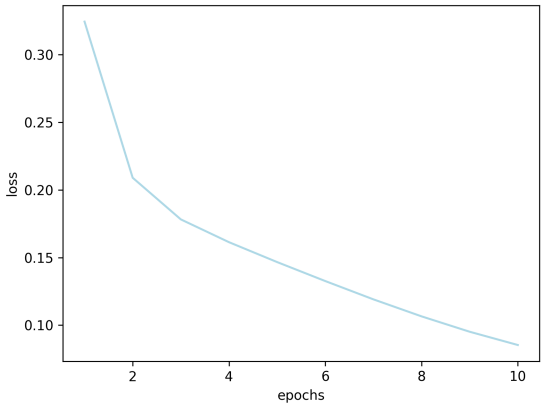

图 9: DeepFM 模型训练损失曲线

由损失的下降曲线得到在训练轮次为 10 时，模型训练虽未完全收敛，但接近收敛状态。

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97501	0.26201	0.41302	45782490
1	0.02500	0.73807	0.04837	1173910
accuracy			0.27391	46956400
macro avg	0.50000	0.50004	0.23069	46956400
weighted avg	0.95126	0.27391	0.40391	46956400

图 10: DeepFM 模型在测试集上效果

DeepFM 模型在测试集上预测准确率为 0.27391，并且对未交互图书的预测精确率和对交互图书的预测召回率高，分别为 0.97501 和 0.73807。

四、调节超参数

（1）激活函数：将 relu 激活函数替换为 tanh 激活函数

	precision	recall	f1-score	support
0	0.97504	0.33310	0.49656	45782490
1	0.02502	0.66738	0.04823	1173910
accuracy			0.34146	46956400
macro avg	0.50003	0.50024	0.27239	46956400
weighted avg	0.95128	0.34146	0.48535	46956400

图 11: 激活函数为 tanh 时，DeepFM 模型在测试集上效果

tanh() 激活函数对总体准确率，未交互图书的预测精确率和召回率，交互图书的预测精确率均有所提升。但是由于计算资源有限，两个模型并不是在完全相同的训练集和测试集下完成的训练和测试，所以不排除这种提升是由于负样本生成的随机性带来的。

（2）网络结构：在该模型的提出论文中，作者提到神经元总个数、每层神经元个数和层与层之间神经元个数形态，例如平直型 [32,32,32]、上升型 [8,16,32]、下降型 [32,16,8] 会对

模型性能造成影响。由于计算资源有限，这里并不能对所有可能的网络结构取值进行探索，仅挑选部分可能结构进行实验。

mlp_dims = [32, 32, 32]：

	precision	recall	f1-score	support
0	0.97505	0.20877	0.34391	45782490
1	0.02501	0.79164	0.04849	1173910
accuracy			0.22334	46956400
macro avg	0.50003	0.50020	0.19620	46956400
weighted avg	0.95130	0.22334	0.33652	46956400

图 12: 网络结构为 32-32-32 时，DeepFM 模型在测试集上效果

对比原模型，在交互图书预测召回率上有存在提升。

mlp_dims = [16, 16, 16]：

	precision	recall	f1-score	support
0	0.97504	0.10923	0.19645	45782490
1	0.02500	0.89095	0.04864	1173910
accuracy			0.12877	46956400
macro avg	0.50002	0.50009	0.12255	46956400
weighted avg	0.95129	0.12877	0.19275	46956400

图 13: 网络结构为 16-16-16 时，DeepFM 模型在测试集上效果

对比原模型及网络结构为 [32,32,32] 时的情况，网络结构 [16,16,16] 对交互图书预测召回率有明显提升。

mlp_dims = [16, 16, 32]：

	precision	recall	f1-score	support
0	0.97498	0.10440	0.18860	45782490
1	0.02500	0.89552	0.04864	1173910
accuracy			0.12418	46956400
macro avg	0.49999	0.49996	0.11862	46956400
weighted avg	0.95123	0.12418	0.18510	46956400

图 14: 网络结构为 16-16-32 时，DeepFM 模型在测试集上效果

结合网络结构为 [16,16,16] 时的效果来看，上升型的网络结构对提高模型效果尤其是对交互图书的召回率似乎有一定作用。

mlp_dims = [32, 16, 8]：

	precision	recall	f1-score	support
0	0.97497	0.16163	0.27730	45782490
1	0.02500	0.83820	0.04854	1173910
accuracy			0.17855	46956400
macro avg	0.49998	0.49991	0.16292	46956400
weighted avg	0.95122	0.17855	0.27158	46956400

图 15: 网络结构为 32-16-8 时，DeepFM 模型在测试集上效果

这个下降型的网络结构在交互图书的召回率上的表现似乎更加印证了上述的猜想。

（3）丢弃率水平

dropout = 0.2：

	precision	recall	f1-score	support
0	0.97499	0.18746	0.31447	45782490
1	0.02500	0.81245	0.04850	1173910
accuracy			0.20309	46956400
macro avg	0.49999	0.49996	0.18148	46956400
weighted avg	0.95124	0.20309	0.30782	46956400

图 16: dropout=0.2 时，DeepFM 模型在测试集上效果

dropout = 0.8：

	precision	recall	f1-score	support
0	0.97493	0.09193	0.16801	45782490
1	0.02499	0.90783	0.04865	1173910
accuracy			0.11232	46956400
macro avg	0.49996	0.49988	0.10833	46956400
weighted avg	0.95119	0.11232	0.16503	46956400

图 17: dropout=0.8 时，DeepFM 模型在测试集上效果

对 dropout 调整的两种情况下的准确率对比原模型均有所降低，丢弃率调整为接近 1 时准确率下降相对更严重。

4.2 Attention Factorization Machine

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）注意力网络维度大小 attn_size：8

（3）丢弃率 dropouts：[0.25, 0.25]

（4）学习率 lr：0.001

二、模型训练过程

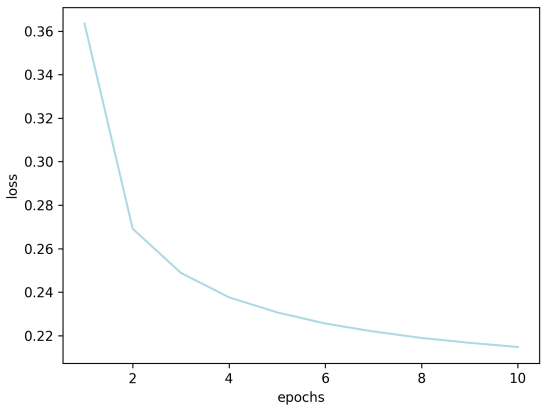

图 18: AFM 模型训练损失曲线

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97497	0.66029	0.78735	45782490
1	0.02493	0.33877	0.04645	1173910
accuracy			0.65225	46956400
macro avg	0.49995	0.49953	0.41690	46956400
weighted avg	0.95121	0.65225	0.76883	46956400

图 19: AFM 模型在测试集上效果

对比 DeepFM 模型和 AFM 模型的训练误差变化曲线，AFM 接近收敛时的训练误差甚至比 DeepFM 大，但 AFM 模型相比 DeepFM 模型对未出现数据的预测效果明显较优。

四、调节超参数

（1）优化器：尝试依据原论文内容，使用随机梯度下降优化方法。

	precision	recall	f1-score	support
0	0.97500	0.85526	0.91121	45782490
1	0.02502	0.14486	0.04267	1173910
accuracy			0.83750	46956400
macro avg	0.50001	0.50006	0.47694	46956400
weighted avg	0.95125	0.83750	0.88950	46956400

图 20: 使用 SGD 优化方法

使用 SGD 优化方法后，模型在测试集上的准确率有较大提升，这主要是由于对未交互图书的召回能力的提升。

（2）丢弃率水平

dropouts = [0.5, 0.5]：

```
              precision    recall  f1-score   support

           0    0.97501   0.79154   0.87375  45782490
           1    0.02504   0.20882   0.04472   1173910

    accuracy                        0.77697  46956400
   macro avg    0.50003   0.50018   0.45924  46956400
weighted avg    0.95126   0.77697   0.85302  46956400
```

图 21: dropouts=[0.5,0.5]

dropouts = [0.75, 0.75]：

```
afm_0.75_0.75 :
              precision    recall  f1-score   support

           0    0.97499   0.85782   0.91266  45782490
           1    0.02494   0.14186   0.04243   1173910

    accuracy                        0.83992  46956400
   macro avg    0.49997   0.49984   0.47754  46956400
weighted avg    0.95124   0.83992   0.89090  46956400
```

图 22: dropouts=[0.75,0.75]

目前尝试的三个 dropouts 水平中，0.75-0.75 水平和结构在测试集中的准确率表现最好，但是对未交互图书的召回率水平对比原模型 0.25-0.25 的结构均有所下降。

（3）正则化方法：依据原论文内容，不使用 dropout 方法防止模型过拟合，而是使用 l2 正则化项。设置优化器 weight_decay 参数为 1e-2。

```
              precision    recall  f1-score   support

           0    0.97500   1.00000   0.98734  45782490
           1    0.00000   0.00000   0.00000   1173910

    accuracy                        0.97500  46956400
   macro avg    0.48750   0.50000   0.49367  46956400
weighted avg    0.95063   0.97500   0.96266  46956400
```

图 23: 使用 l2 正则化项

可以看出这个训练出现了一些问题，生成所有预测结果均为 0，我猜测可能是由于 λ 设置地可能有点大导致的。

4.3 Deep Cross Network

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）交叉层层数 num_layers：3

（3）前馈神经网络层各层维度 mlp_dims：[16, 16]

（4）丢弃率 dropout：0.2

（5）学习率 lr：0.001

二、模型训练过程

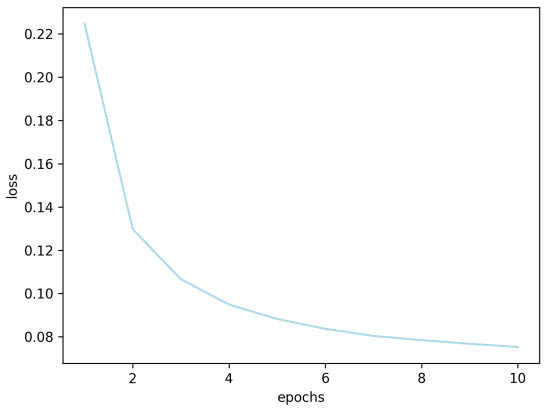

图 24: DCN 模型训练损失曲线

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97510	0.17247	0.29309	45782490
1	0.02502	0.82827	0.04858	1173910
accuracy			0.18886	46956400
macro avg	0.50006	0.50037	0.17083	46956400
weighted avg	0.95135	0.18886	0.28698	46956400

图 25: DCN 模型在测试集上效果

四、调节超参数

探究交叉层层数 num_layers 对模型效果的影响：

num_layers = 2：

	precision	recall	f1-score	support
0	0.97497	0.20888	0.34404	45782490
1	0.02499	0.79085	0.04845	1173910
accuracy			0.22343	46956400
macro avg	0.49998	0.49986	0.19625	46956400
weighted avg	0.95122	0.22343	0.33665	46956400

图 26: 交叉网络层数为 2

num_layers = 4：

	precision	recall	f1-score	support
0	0.97493	0.21505	0.35237	45782490
1	0.02498	0.78435	0.04842	1173910
accuracy			0.22928	46956400
macro avg	0.49996	0.49970	0.20039	46956400
weighted avg	0.95118	0.22928	0.34477	46956400

图 27: 交叉网络层数为 4

num_layers = 5：

	precision	recall	f1-score	support
0	0.97501	0.16468	0.28177	45782490
1	0.02500	0.83536	0.04855	1173910
accuracy			0.18144	46956400
macro avg	0.50000	0.50002	0.16516	46956400
weighted avg	0.95126	0.18144	0.27593	46956400

图 28: 交叉网络层数为 5

随着模型复杂度的增加，在测试集上的准确率大体呈现先增加后减少的趋势。

4.4 Automatic Feature Interaction Learning

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）头数 head_num：2

（3）多头注意力层层数 attn_layers：3

（4）前馈神经网络层各层维度 mlp_dims：[32, 16]

（5）丢弃率 dropout：0.2

（6）学习率 lr：0.001

二、模型训练过程

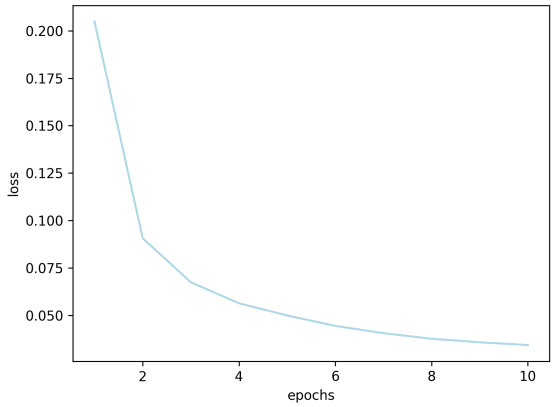

图 29: AutoInt 模型训练损失曲线

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97502	0.12520	0.22191	45782490
1	0.02500	0.87491	0.04862	1173910
accuracy			0.14394	46956400
macro avg	0.50001	0.50006	0.13526	46956400
weighted avg	0.95127	0.14394	0.21758	46956400

图 30: AutoInt 模型在测试集上效果

四、调节超参数

（1）头数 head_num

head_num = 3：

	precision	recall	f1-score	support
0	0.97497	0.15808	0.27206	45782490
1	0.02499	0.84173	0.04855	1173910
accuracy			0.17517	46956400
macro avg	0.49998	0.49991	0.16030	46956400
weighted avg	0.95122	0.17517	0.26647	46956400

图 31: 多头注意力层头数为 3

head_num = 4：

	precision	recall	f1-score	support
0	0.97497	0.13393	0.23550	45782490
1	0.02499	0.86588	0.04859	1173910
accuracy			0.15223	46956400
macro avg	0.49998	0.49991	0.14205	46956400
weighted avg	0.95122	0.15223	0.23083	46956400

图 32: 多头注意力层头数为 4

head_num = 5：

	precision	recall	f1-score	support
0	0.97508	0.14971	0.25957	45782490
1	0.02501	0.85076	0.04860	1173910
accuracy			0.16724	46956400
macro avg	0.50005	0.50024	0.15409	46956400
weighted avg	0.95132	0.16724	0.25430	46956400

图 33: 多头注意力层头数为 5

当多头注意力层头数为 3 时，模型在测试集上的准确率表现最好，为 0.17517。随着头数的增加，模型在测试集上的准确率大体呈现一个先增加后减少的趋势。

（2）多头注意力层层数 attn_layers

attn_layers = 4：

	precision	recall	f1-score	support
0	0.97495	0.17268	0.29340	45782490
1	0.02499	0.82698	0.04851	1173910
accuracy			0.18904	46956400
macro avg	0.49997	0.49983	0.17096	46956400
weighted avg	0.95120	0.18904	0.28728	46956400

图 34: 多头注意力层层数为 4

attn_layers = 5：

	precision	recall	f1-score	support
0	0.97493	0.15120	0.26179	45782490
1	0.02499	0.84836	0.04854	1173910
accuracy			0.16863	46956400
macro avg	0.49996	0.49978	0.15517	46956400
weighted avg	0.95118	0.16863	0.25646	46956400

图 35: 多头注意力层层数为 5

attn_layers = 6：

	precision	recall	f1-score	support
0	0.97493	0.14846	0.25768	45782490
1	0.02499	0.85111	0.04855	1173910
accuracy			0.16603	46956400
macro avg	0.49996	0.49979	0.15312	46956400
weighted avg	0.95118	0.16603	0.25245	46956400

图 36: 多头注意力层层数为 6

当多头注意力层层数为 3 时，模型在测试集上的准确率表现最好，为 0.18904。

4.5 Deep Interest Network

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）前馈神经网络层各层维度 mlp_dims：[64, 32]

（3）丢弃率 dropout：0.5

（4）学习率 lr：0.001

二、模型训练过程

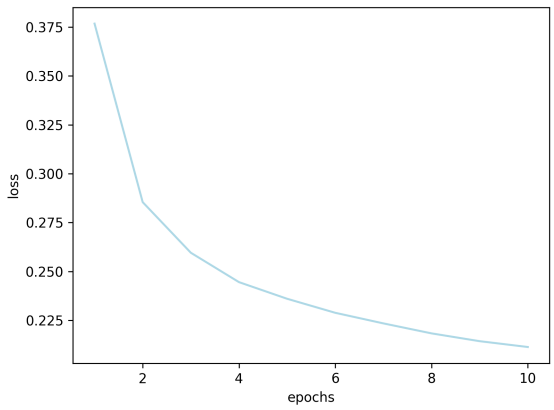

图 37: DIN 模型训练损失曲线

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97502	0.68355	0.80367	45782490
1	0.02504	0.31695	0.04641	1173910
accuracy			0.67438	46956400
macro avg	0.50003	0.50025	0.42504	46956400
weighted avg	0.95127	0.67438	0.78474	46956400

图 38: DIN 模型在测试集上效果

四、调节超参数

调节丢弃率 dropout 水平

dropout = 0.5：

	precision	recall	f1-score	support
0	0.97500	0.67915	0.80061	45782490
1	0.02499	0.32074	0.04637	1173910
accuracy			0.67018	46956400
macro avg	0.49999	0.49994	0.42349	46956400
weighted avg	0.95125	0.67018	0.78176	46956400

图 39: DIN 模型 dropout=0.5

dropout = 0.8：

	precision	recall	f1-score	support
0	0.97500	0.89419	0.93285	45782490
1	0.02504	0.10598	0.04051	1173910
accuracy			0.87449	46956400
macro avg	0.50002	0.50008	0.48668	46956400
weighted avg	0.95126	0.87449	0.91054	46956400

图 40: DIN 模型 dropout=0.8

当 dropout 水平为 0.8 时，模型在测试集上的准确率提升显著，为 0.87449。

4.6 Dual Input-aware Factorization Machine

一、模型参数及超参数

（1）嵌入层维度 embed_dim：8

（2）注意力头数 header_num：2

（3）前馈神经网络层各层维度 mlp_dims：[128, 64]

（4）丢弃率 dropout：0.5

（5）学习率 lr：0.001

二、模型训练过程

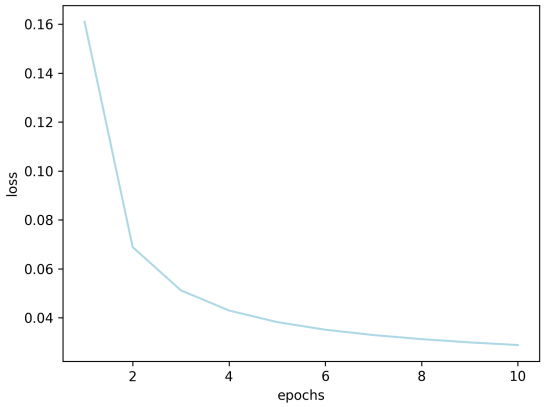

图 41: DIFM 模型训练损失曲线

三、在测试集上的表现

	precision	recall	f1-score	support
0	0.97509	0.11986	0.21348	45782490
1	0.02501	0.88056	0.04864	1173910
accuracy			0.13888	46956400
macro avg	0.50005	0.50021	0.13106	46956400
weighted avg	0.95133	0.13888	0.20936	46956400

图 42: DIFM 模型在测试集上效果

四、调节超参数

调整多头注意力网络头数 header_num 对模型效果的影响。

header_num = 3：

	precision	recall	f1-score	support
0	0.97495	0.11717	0.20920	45782490
1	0.02499	0.88261	0.04861	1173910
accuracy			0.13631	46956400
macro avg	0.49997	0.49989	0.12891	46956400
weighted avg	0.95121	0.13631	0.20518	46956400

图 43: DIFM 模型中注意力头数为 3

header_num = 4：

	precision	recall	f1-score	support
0	0.97492	0.11556	0.20663	45782490
1	0.02499	0.88406	0.04861	1173910
accuracy			0.13477	46956400
macro avg	0.49995	0.49981	0.12762	46956400
weighted avg	0.95117	0.13477	0.20268	46956400

图 44: DIFM 模型中注意力头数为 4

header_num = 5：

	precision	recall	f1-score	support
0	0.97505	0.12473	0.22117	45782490
1	0.02501	0.87553	0.04863	1173910
accuracy			0.14350	46956400
macro avg	0.50003	0.50013	0.13490	46956400
weighted avg	0.95130	0.14350	0.21685	46956400

图 45: DIFM 模型中注意力头数为 5

从模型在测试集的变现来看，注意力层头数的改变对模型效果的影响比较微小。

5 模型评估

5.1 准确率、精确率、召回率、F1 分数

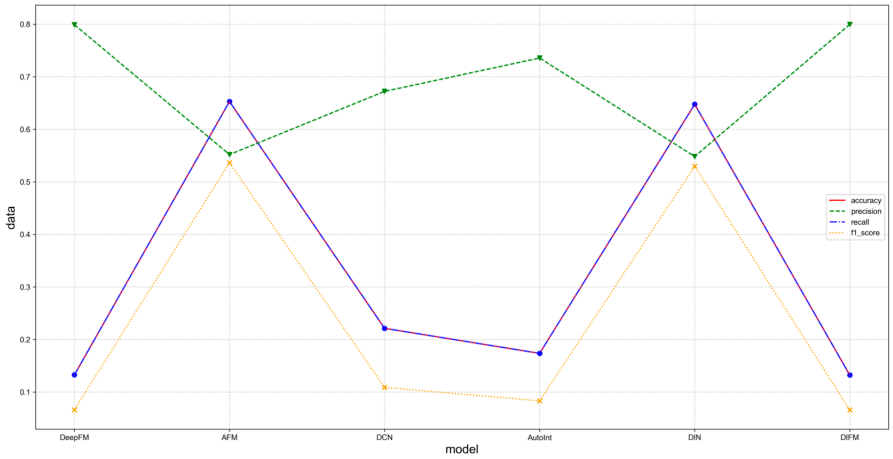

图 46: 评估曲线

5.2 混淆矩阵

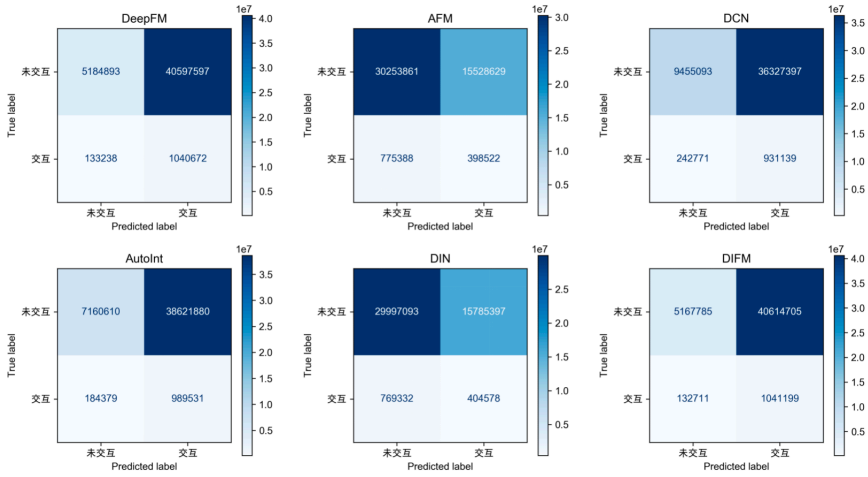

图 47: 混淆矩阵

6 反思总结

6.1 负样本构建

(1) 训练集中负样本构造比例应该是本问题的超参数之一，选择一个合适的正负样本的比例直觉上有助于提升模型的效果。

（2）测试集中负样本构造比例应该也是本问题的超参数之一，在本实验中直觉地将测试集构建得相比训练集更加稀疏。但实际上到底是与训练集分布相同或者比训练集稀疏程度为多少时，测试集最能够反映模型效果的问题仍有待商榷，需要进行更多的实验说明。

（3）本实验中负样本构建的随机规则类似于自助法采样，是有放回的抽取，所以会导致部分内容重合，并且部分内容有一定概率无法采样到。

6.2 超参数选择

（1）感觉似乎进行超参数选择最大的阻碍是有限的时间和计算资源。如果只是用 CPU 进行训练，某些模型一个 epoch 就需要运行 5 个小时。在 GPU 的加持下，完成一次完整训练的时间缩短至平均一小时左右，但仍然需要大量时间。而且线上计算资源 colab 十分不稳定，经常容易断开连接，kaggle 的 GPU 计算资源每周限制使用 30h。

（2）虽说是超参数选择，我感觉更像是探究超参数对模型的影响。因为并不是遍历超参数组合而是分别改变单个超参数的取值，观察模型效果。这点是需要改进的。

6.3 评价指标

（1）官方 baseline 中采用的是在为测试集每个正样本生成 99 个负样本列表的前提下，对正样本的预测概率在这 100 个样本中排名前 20，是命中率。

（2）在本实验中，只有预测概率大于 50% 才认为是用户交互的图书，这个条件是不是过于严格了。如果是 baseline 中的命中率评估思想，那么预测概率不一定达到这个阈值也可以被认为是预测准确的。

（3）基于现实生活中的推荐系统场景，在用户物品交互极为稀疏的大环境下，对正样本预测的精确率和召回率是比较重要的。精确率描述推荐系统向用户推送可能感兴趣物品时能不能准确戳中用户的喜好，召回率描述推荐系统挖掘用户所有感兴趣物品的程度。